LAB 1

AIM – Analysis and Identification of suitable process model for your project.

TEAM MEMBERS :

ARYAN RAJ (24BDS1094)

YASH RAJ (24BDS1209)

4 Candidate Software Development Models

- **Agile Model**: An iterative and incremental process framework that emphasizes short development cycles (sprints), continuous stakeholder collaboration, early delivery of functional software, and high adaptability to changing requirements.
- **Incremental Model**: A methodology where software requirements are divided into multiple standalone, prioritized modules. Each module undergoes sequential requirement analysis, design, implementation, and testing before being integrated into the core system build.
- **Spiral Model**: A risk-driven meta-model combining iterative prototyping with structured sequential control. Development moves through repeating cycles of objective determination, risk identification and mitigation, engineering, and evaluation.
- **Rapid Application Development (RAD) Model**: A high-velocity sequential process that relies on component-based construction, quick prototyping, automated code generation, and strict time-boxed schedules.

Chosen SDLC Model: Agile Model

Agile Model is chosen as the best software development life cycle for the MSME Multi-Vertical Commodity Marketplace project.

Comparative Justification Against Optioned Models

- **Agile vs. Incremental Model (Handling Dynamic Plug-in Logic Across Verticals)**:
  Project Requirement: The platform supports eight disparate commodity sectors (Agricultural produce, Textiles, Chemicals, Plastics, etc.), each requiring distinct grading parameters, units of measurement, and pricing rules managed dynamically through backend configuration schema (VerticalConfig).
  Justification: The Incremental Model assumes pre-defined, static feature boundaries for each planned release. In contrast, Agile enables building a single, generic core engine (grading, pricing, allocation) in early sprints and plugging in vertical specific configurations sprint-by-sprint based on user testing, avoiding rigid architectural lock-in.
- **Agile vs. Spiral Model (Managing AI Verification & Development Overhead)**:
  Project Requirement: The system incorporates an AI-assisted grading model with confidence scoring and a human verifier fallback mechanism to handle low-confidence uploads.
  Justification: While the Spiral Model manages high risks through continuous prototyping, it requires extensive risk-analysis documentation, specialized evaluation expertise, and heavy administrative overhead. For a college-level project, Agile provides the necessary feedback loops to tune AI thresholds and verifier workflows incrementally without the prohibitive managerial complexity of Spiral.
- **Agile vs. RAD Model (Managing Unfinalized Integrations & Multi-Platform Stack)**:
  Project Requirement: Essential system modules—including market price data feeds, delivery handoffs, payment gateways, and media storage—are deliberately unfinalized and open for research. Simultaneously, the project spans a FastAPI backend, PostgreSQL database, web portal, and Kotlin mobile app.
  Justification: The RAD Model requires complete initial requirement clarity, rigid schedule constraints, and full availability of pre-built reusable components across all modules simultaneously. Agile allows building the core API and data layer first while deferring unfinalized external integrations to future sprint backlogs as research progresses.
- **Agile for Multi-Seller Order Allocation Complexity**:

Project Requirement: The platform features an order matching engine that splits a single buyer requirement across multiple seller listings (OrderAllocation) while factoring in region, price, and reputation scores (USER).
Justification: Agile sprints allow step-by-step implementation, isolated unit testing, and continuous optimization of complex allocation algorithms before full system deployment.

Phases of the Agile Model

1. **Requirements & Backlog Grooming**: Defining user roles (Seller, Buyer, Admin, Verifier) and decomposing major feature systems (listing creation, AI grading, multi-seller allocation) into user stories within the product backlog.
2. **Sprint Planning**: Selecting high-priority backlog items to commit to during a 1-to-2-week development cycle.
3. **Sprint Execution (Design & Coding)**: Concurrently developing FastAPI backend endpoints, PostgreSQL database schemas, and frontend/mobile UI components for committed user stories.
4. **Testing & Integration**: Running continuous unit, integration, and UI tests on completed features prior to code merging.
5. **Sprint Review & Stakeholder Demonstration**: Demonstrating functional software increments to gather feedback and refine remaining product backlog priorities.
6. **Retrospective & Incremental Deployment**: Reviewing team performance, optimizing workflows, and deploying stable software builds at the end of each sprint cycle.